Building a Simple Pay-Later Service

1 Overview

As a pay later service, we allow our users to buy goods from a merchant now, and then allow them to pay for those goods at a later date.

The service operates within the following constraints:

- For every transaction paid through us, we deduct a transaction fee while paying back to the merchant.
  - For example, if the transaction amount is Rs.100, and the transaction fee is 10%, we pay Rs. 90 back to the merchant.
  - The fee varies from merchant to merchant.
  - A merchant can decide to change the fee at any point in time.
- All users are onboarded with a credit limit, beyond which they cannot transact.
  - If a transaction value exceeds this credit limit, we reject the transaction.

1.1 Use Cases

The service is intended to fulfill the following use cases:

- Allow merchants to be onboarded with a fee.
- Allow merchants to change their fee.
- Allow users to be onboarded (name, email-id, and credit limit).
- Allow a user to carry out a transaction of some amount with a merchant.
- Allow a user to pay back their dues (full or partial).
- Reporting:
  - How much fee weve collected from a merchant to date.
  - Dues for a user so far.
  - Which users have reached their credit limit.
  - Total dues from all users together.

2 Goal

The goal of this coding challenge is to build a backend web system to satisfy the above use cases.

- Input/Output will be via HTTP API endpoints using FastAPI in Python.
- The input can be given in any order as API requests, and the system should respond accordingly.
- For inputs like merchant fee percentage changes or credit limit changes for a user, the system adapts itself.
- Use MongoDB as the database for storing users, merchants, transactions, and related data.

2.1 API

Here is how the HTTP API endpoints, corresponding to the use cases mentioned above, can look like (using FastAPI conventions for routes and request bodies):

- `POST /users`
  - Request body: `{"name": "u1", "email": "u1@email.in", "credit_limit": 1000}`
  - Response: `{"name": "u1", "credit_limit": 1000}`
- `POST /merchants`
  - Request body: `{"name": "m1", "fee_percentage": 2.0}`
  - Response: `{"name": "m1", "fee_percentage": 2.0}`
- `POST /transactions`
  - Request body: `{"user_name": "u1", "merchant_name": "m2", "amount": 400}`
  - Response: `{"status": "success"}` or `{"status": "rejected", "reason": "credit limit"}`
- `PATCH /merchants/{merchant_name}`
  - Request body: `{"fee_percentage": 1.0}`
  - Response: `{"name": "m1", "fee_percentage": 1.0}`
- `POST /paybacks`
  - Request body: `{"user_name": "u1", "amount": 300}`
  - Response: `{"user_name": "u1", "remaining_dues": 100}`
- `GET /reports/fee/{merchant_name}`
  - Response: `{"fee_collected": 10.5}`
- `GET /reports/dues/{user_name}`
  - Response: `{"dues": 200}`
- `GET /reports/users-at-credit-limit`
  - Response: `["user1", "user3"]`
- `GET /reports/total-dues`
  - Response: `{"total": 400, "details": {"user1": 300, "user3": 100}}`

3 Example Flow

```
> POST /users {"name": "user1", "email": "u1@users.com", "credit_limit": 300}
{"name": "user1", "credit_limit": 300}

> POST /users {"name": "user2", "email": "u2@users.com", "credit_limit": 400}
{"name": "user2", "credit_limit": 400}

> POST /users {"name": "user3", "email": "u3@users.com", "credit_limit": 500}
{"name": "user3", "credit_limit": 500}

> POST /merchants {"name": "m1", "fee_percentage": 0.5}
{"name": "m1", "fee_percentage": 0.5}

> POST /merchants {"name": "m2", "fee_percentage": 1.5}
{"name": "m2", "fee_percentage": 1.5}

> POST /merchants {"name": "m3", "fee_percentage": 1.25}
{"name": "m3", "fee_percentage": 1.25}

> POST /transactions {"user_name": "user2", "merchant_name": "m1", "amount": 500}
{"status": "rejected", "reason": "credit limit"}

> POST /transactions {"user_name": "user1", "merchant_name": "m2", "amount": 300}
{"status": "success"}

> POST /transactions {"user_name": "user1", "merchant_name": "m3", "amount": 10}
{"status": "rejected", "reason": "credit limit"}

> GET /reports/users-at-credit-limit
["user1"]

> POST /transactions {"user_name": "user3", "merchant_name": "m3", "amount": 200}
{"status": "success"}

> POST /transactions {"user_name": "user3", "merchant_name": "m3", "amount": 300}
{"status": "success"}

> GET /reports/users-at-credit-limit
["user1", "user3"]

> GET /reports/fee/m3
{"fee_collected": 6.25}

> POST /paybacks {"user_name": "user3", "amount": 400}
{"user_name": "user3", "remaining_dues": 100}

> GET /reports/total-dues
```

```
{"details": {"user1": 300, "user3": 100}, "total": 400}
```

4 Tips

- We appreciate and look for good quality code.
- Having good coding standards, with proper object-oriented design and responsibility segregation will go a long way in impressing us.
- Tests are important.
- Great to have: An extensible and flexible code-base that wont require a redesign to address possible future requirements in the given theme.
- Make assumptions where you need to.
- Security aspects need not be considered to limit the scope.
- You must use Python with FastAPI for the backend API and MongoDB as the database. No other tech stack is allowed.
- Code submission can be done via email to HR with a zip file. Do not create any public repository on any VCS.